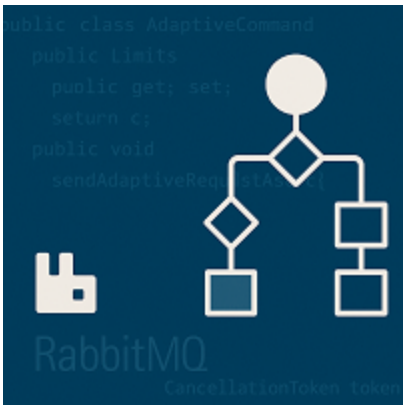


⚡ Adaptive Test in C#

Reference: RA 475

This package provides the core classes for managing adaptive test configuration requests in automated test equipment (ATE) environments. Enable dynamic control of test steps and enable words via well-structured messages sent over RabbitMQ to the AMP server.

Purpose

The goal of `Teradyne.FA.DIA.DAS.Adaptive` is to provide a standardized and easy-to-integrate API for:

- Constructing valid adaptive configuration requests (`AdaptiveCommand`)
- Sending those requests asynchronously via RabbitMQ (`AdaptiveCommunication`)
- Managing adaptive logic in a modular and testable way

Dependencies

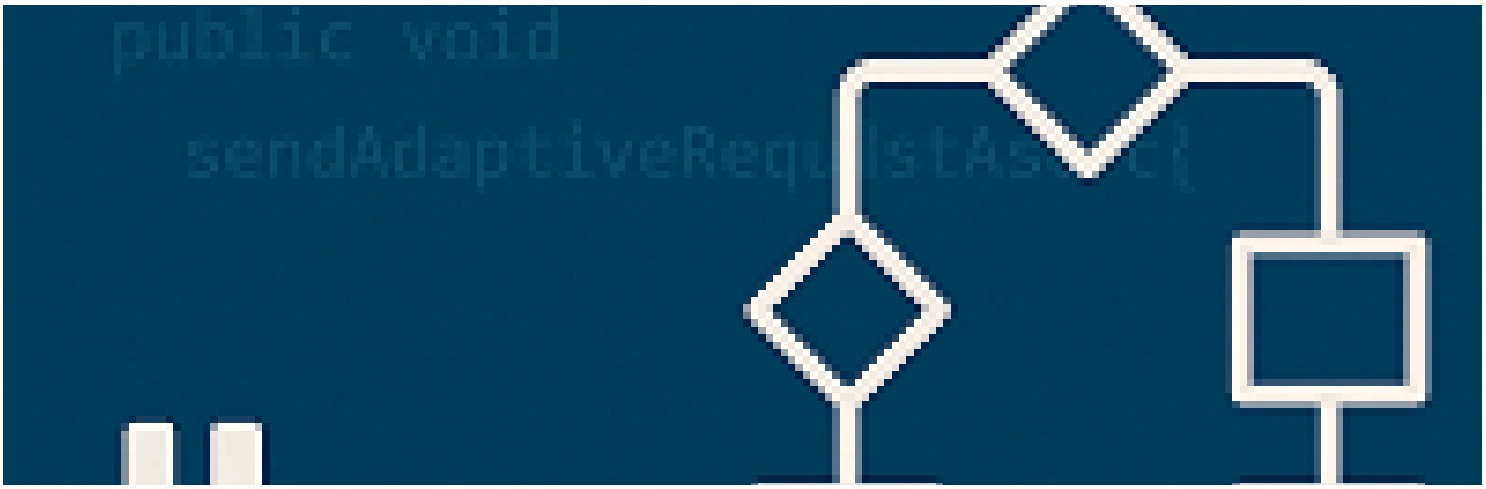
Dependency	Purpose
`RabbitMQ.Client`	Manages AMQP communication with the AMP server

Typical Use Case

1. Create an `AdaptiveCommand` instance to define your adaptive behavior (enable word, test instance, limits...).
2. Wrap it into an `AdaptiveRequest` and set the request target (`AdaptiveRequestTarget`).
3. Send it using the `AdaptiveCommunication` class, which handles the RabbitMQ exchange.

Features

- Enable/disable test elements dynamically.
- Update test limits on-the-fly.
- Strongly typed and well-structured commands.



Advanced Usage – Adaptive Test Commands

Overview

This module enables adaptive communication with test programs using RabbitMQ. The typical flow includes:

1. Constructing an `AdaptiveCommand`.
2. Wrapping it in an `AdaptiveRequest`.
3. Sending it using `AdaptiveCommunication`.

Class Structure

`AdaptiveCommand.cs`

Represents the command to send adaptively to the test equipment.

```
public class AdaptiveCommand
{
    public EnableWordAction? EnableWordAction { get; set; }
    public TestStepAction? TestStepAction { get; set; }
    public Limits? Limits { get; set; }
}
```

`EnableWordAction.cs`

```
public class EnableWordAction
{
    public string EnableWordName { get; set; }
    public EnableWordActionType ActionType { get; set; }
}
```

Limits.cs

```
public class Limits
{
    public string TestInstanceName { get; set; }
    public string ParamName { get; set; }
    public double? Low { get; set; }
    public double? High { get; set; }
}
```

AdaptiveRequest.cs

```
public class AdaptiveRequest
{
    public AdaptiveRequestTarget RequestTarget { get; set; }
    public AdaptiveCommand AdaptiveCommand { get; set; }
}
```

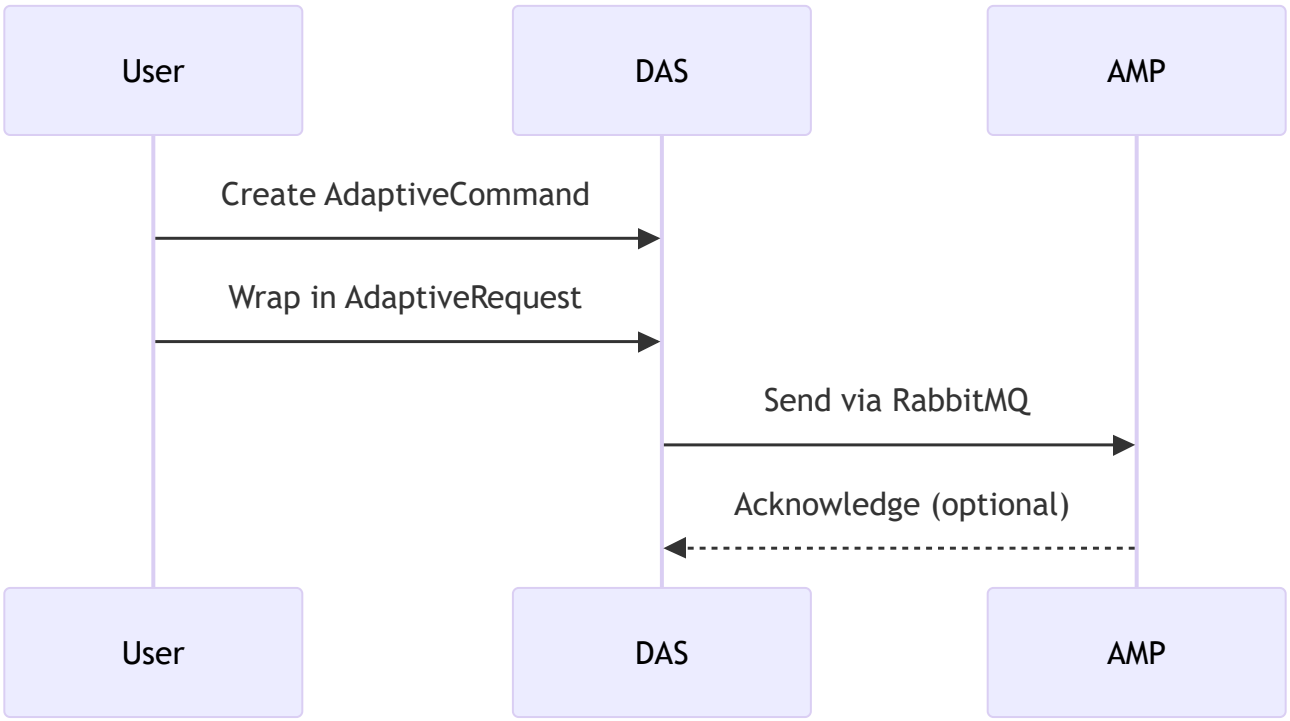
AdaptiveRequestTarget.cs

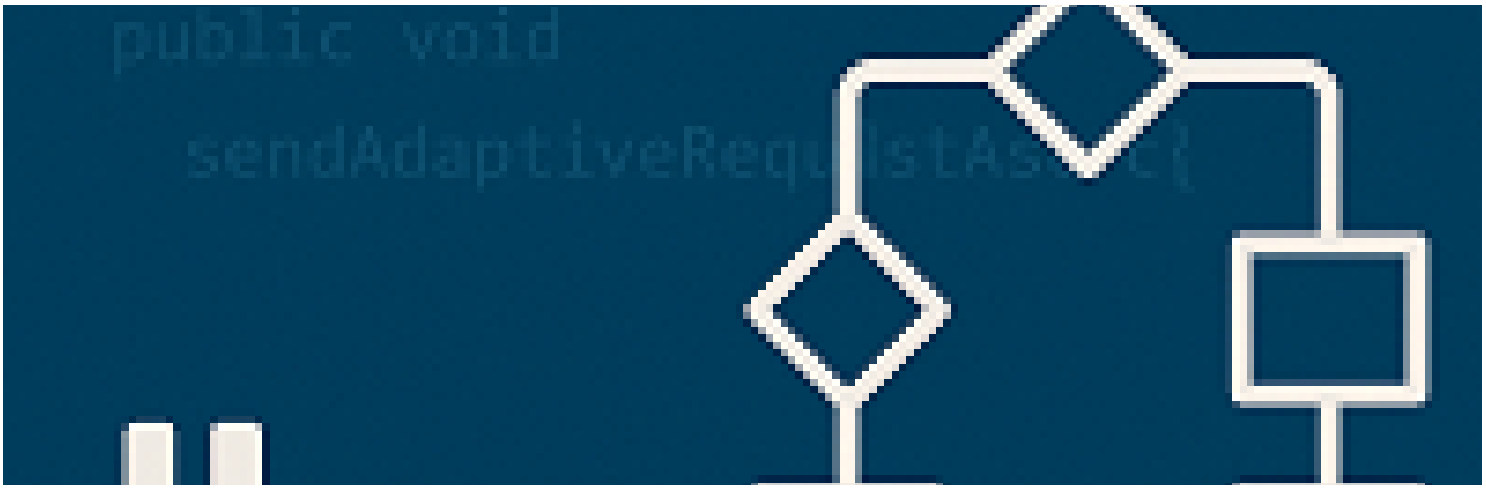
```
public class AdaptiveRequestTarget
{
    public string LotId { get; set; }
    public string TesterId { get; set; }
    public string TestCell { get; set; }
}
```

AdaptiveCommunication.cs

```
public class AdaptiveCommunication : ObservableObjectLogging
{
    public async Task SendAdaptiveRequestAsync(AdaptiveRequest request,
CancellationToken token)
    {
        // Serialize to JSON and publish to RabbitMQ
    }
}
```

Sequence Diagram


User
DAS
AMP
Create AdaptiveCommand
Wrap in AdaptiveRequest
Send via RabbitMQ
Acknowledge (optional)
User
DAS
AMP



RabbitMQ Connection Parameters

The `AdaptiveCommunication` class uses the following constants to establish a connection with RabbitMQ:

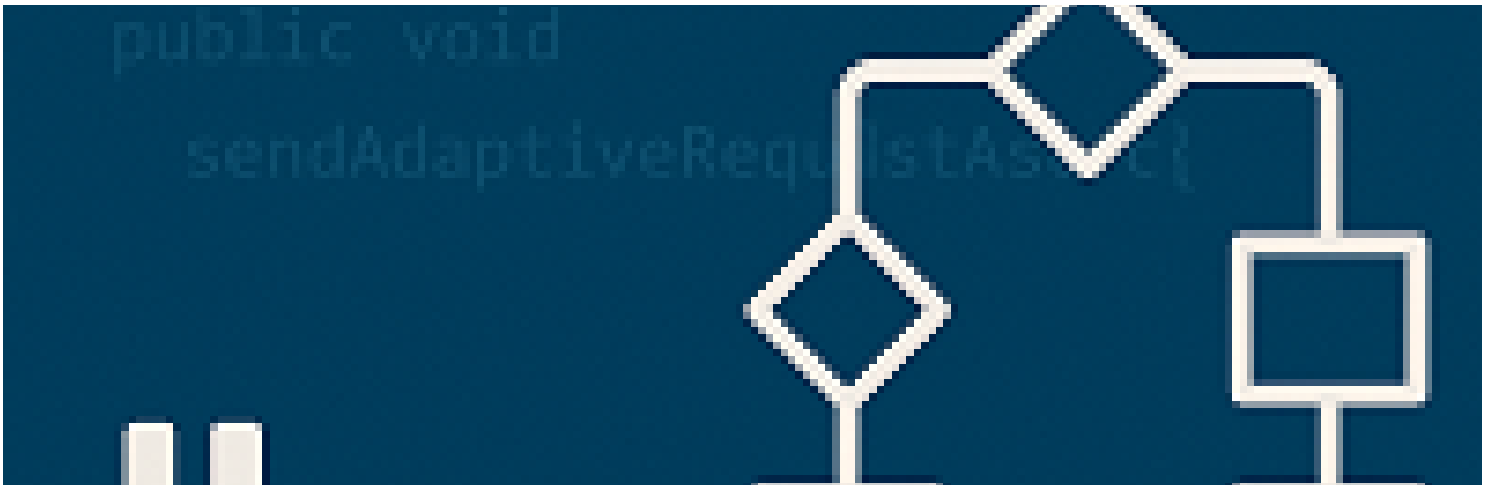
Parameter	Value	Description
`RMQ_PORT`	`5672`	Default AMQP port
`RMQ_SCHEME`	`amqp://`	RabbitMQ connection scheme
`RMQ_USER`	`AMP-adaptive`	Username for authentication
`RMQ_PWD`	`TER-ADAPTIVE-P4SSW0RD`	Password for authentication (plaintext) ⚠
`RMQ_VHOST`	`TER-AMP-vhost`	Virtual host on the RabbitMQ server
`RMQ_EXCHANGE`	`AMP_DAS_Exchange`	Exchange to publish messages
`RMQ_QUEUE`	`AMP_DAS_Queue`	Default queue for responses
`RMQ_ROUTING`	`AMP_DAS_RoutingKey`	Routing key for the message
`RMQ_CLIENTNAME`	`AMP DAS RabbitMQ Sender`	Used for logging/debugging client identifier

Dynamic Parameter

```
public AdaptiveCommunication(string serverIP)
```

⚠ Security Note

> The password is currently stored in plaintext in the source code. For production environments, consider using a secure vault or secrets manager.



How to Send an Adaptive Command

This guide explains how to use an interactive C# console application to manage Adaptive Test commands using the `Teradyne.FA.DIA.DAS.Adaptive` library.

Key Features

- Clear previously issued adaptive commands (excluding enable words)
- Enable or disable individual test steps
- Enable or disable enable words
- Update test limits (low/high)
- Menu-driven interface for rapid experimentation

> **Important:** The following examples send only one command per function. However, it is possible to combine multiple requests into a single command.

Initialization Phase

Before sending any command, you must:

- Specify the origin of the command using a DAS identifier (`dasId`).
- Establish communication with the RabbitMQ server. In the example below, the Adaptive Test runs on `localhost`.

```
public string dasId = "http://127.0.0.1:3000/tems";
public AdaptiveCommunication adaptivecomm = new AdaptiveCommunication("127.0.0.1");
```

Clear Command

The `ClearAdaptiveCommand` function resets all previously issued adaptive test actions—except enable word modifications.

```
public void ClearAdaptiveCommand()
{
    AdaptiveCommand adaptiveCommand = new AdaptiveCommand(dasId);
    adaptiveCommand.addAction(adaptiveCommand.createClearAction());
    string rsp = adaptivecomm.SendAction(adaptiveCommand);
    Console.WriteLine("Clear command Response = " + rsp);
}
```

Enable Test

This command enables a test using its name. (It is also possible to use the test number.)

```
public void EnableTest(string name)
{
    AdaptiveCommand adaptiveCommand = new AdaptiveCommand(dasId);

adaptiveCommand.addAction(adaptiveCommand.createTestStepAction(TestStepActionType.ENABLE,
new List<string>() { name }));
    string rsp = adaptivecomm.SendAction(adaptiveCommand);
    Console.WriteLine($"Enable Test {name} command Response = " + rsp);
}
```

Disable Test

Similar to the enable function, but disables the specified test. You can also disable multiple tests at once by providing multiple names or test numbers.

```
public void DisableTest(string name)
{
    AdaptiveCommand adaptiveCommand = new AdaptiveCommand(dasId);

adaptiveCommand.addAction(adaptiveCommand.createTestStepAction(TestStepActionType.DISABLE,
new List<string>() { name }));
    string rsp = adaptivecomm.SendAction(adaptiveCommand);
    Console.WriteLine($"Disable Test {name} command Response = " + rsp);
}
```

Enable Word Management

This function allows enabling or disabling one or more enable words.

```
public void EnableWord(string name, bool status)
{
    AdaptiveCommand adaptiveCommand = new AdaptiveCommand(dasId);
    adaptiveCommand.addAction(adaptiveCommand.createEnableWordAction(
        status ? EnableWordActionType.ENABLE : EnableWordActionType.DISABLE,
        new List<string>() { name }));
    string rsp = adaptivecomm.SendAction(adaptiveCommand);
    Console.WriteLine($"Enable Word {name} = {status} command Response = " + rsp);
}
```

Limits Management

This function changes the limit values for a test using its name or number. Both `low limit` and `high limit` must be specified.

```
public void SetNewLimit(string name, double lowlimit, double highlimit)
{
    AdaptiveCommand adaptiveCommand = new AdaptiveCommand(dasId);
    adaptiveCommand.addAction(adaptiveCommand.createLimitsAction(
        TestStepActionType.UPDATE_LIMITS,
        new List<string>() { name },
        new Limits() { HI_LIMIT = highlimit, LO_LIMIT = lowlimit, LIMIT_NAMES = new
List<string>() { name } }));
    string rsp = adaptivecomm.SendAction(adaptiveCommand);
    Console.WriteLine($"Set New Limit for Test {name} - Low: {lowlimit}, High: {highlimit} |
Response = " + rsp);
}
```

Close

To cleanly terminate the connection with the RabbitMQ server, call the `Close` method:

```
public void Close()
{
    adaptivecomm.Close();
    Console.WriteLine("Communication Closed");
}
```

Namespace Teradyne.FA.DIA.DAS.Adaptive

Classes

AdaptiveCommand

Builds and serializes AdaptiveRequests to JSON for transmission.

AdaptiveCommunication

This class manages RabbitMQ communication to send adaptive request to the tester host.

AdaptiveRequest

Represents an adaptive request sent to the ATE system, including test step actions, enable word changes, and target information.

EnableWordAction

Represents an action applied to a set of enable words for adaptive behavior.

Limits

Defines numerical limits and metadata associated with a test step or limit configuration.

TestStepAction

Represents an action that can be applied to one or more test steps in an adaptive test scenario.

Enums

AdaptiveRequestTarget

Enum specifying the target component for an adaptive request.

EnableWordActionType

Specifies the type of action to apply on enable words in adaptive control.

TestStepActionType

Defines the types of actions that can be taken on test steps in an adaptive scenario.